

PLOTLY

What is Plotly?

Plotly is a Python library used to create **interactive graphs and charts**.

Unlike Matplotlib, Plotly charts allow users to:

- Zoom in and out
 - Hover over data points
 - Pan the graph
 - Save charts as images
 - Display interactive visualizations in Jupyter Notebook and web applications like Streamlit
-

WHY USE PLOTLY?

Advantages

- Interactive charts
 - Beautiful default themes
 - Easy to use
 - Supports 2D and 3D graphs
 - Works well with Pandas
 - Excellent for dashboards
 - Integrates with Streamlit and Dash
-

Installation

```
pip install plotly
```

Check the version:

```
import plotly
```

```
print(plotly.__version__)
```

Plotly Modules

Module	Purpose
plotly.express	Simple plotting
plotly.graph_objects	Advanced customization
plotly.subplots	Multiple plots
plotly.io	Save and export charts

General Syntax

```
import plotly.express as px
```

```
fig = px.chart_name(  
    data_frame,  
    x=,  
    y=,  
    color=,  
    size=,  
    symbol=,  
    text=,  
    hover_name=,  
    hover_data=,  
    facet_row=,  
    facet_col=,  
    animation_frame=,  
    animation_group=,  
    title=,  
    labels=,  
    width=,  
    height=,  
    template=  
)
```

```
fig.show()
```

Example Dataset

```
import pandas as pd
```

```
data = {  
    "Name":["A","B","C","D","E"],  
    "Age":[20,21,22,23,24],  
    "Marks":[80,70,90,75,85],
```

```
"City":["Delhi","Mumbai","Delhi","Noida","Mumbai"],  
"Gender":["Male","Female","Male","Female","Male"]  
}
```

```
df = pd.DataFrame(data)
```

Common Attributes

1. data_frame

Specifies the dataset.

```
px.scatter(data_frame=df, x="Age", y="Marks")
```

or

```
px.scatter(df, x="Age", y="Marks")
```

2. x

Column for the X-axis.

```
x="Age"
```

3. y

Column for the Y-axis.

```
y="Marks"
```

4. color

Groups data by color.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    color="City"  
)
```

Output

Delhi → Blue

Mumbai → Red

Noida → Green

5. size

Controls bubble size.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    size="Marks"  
)
```

Higher marks → Bigger bubble.

6. symbol

Changes marker shape.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    symbol="Gender"  
)
```

Output

Male → Circle

Female → Diamond

7. text

Displays text on each point.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    text="Name"  
)
```

Output

A

B

C
D

appears on the graph.

8. hover_name

Displays the selected column when hovering.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    hover_name="Name"  
)
```

9. hover_data

Shows extra information on hover.

```
fig = px.scatter(  
    df,  
    x="Age",  
    y="Marks",  
    hover_data=["City", "Gender"]  
)
```

Hover Output

Age:20

Marks:80

City:Delhi

Gender:Male

10. title

Chart title.

```
title="Student Performance"
```

11. width

Chart width.

width=800

12. height

Chart height.

height=500

13. labels

Rename axis labels.

```
labels={  
  "Age": "Student Age",  
  "Marks": "Exam Marks"  
}
```

14. template

Changes chart theme.

template="plotly_dark"

Available templates

plotly

plotly_dark

ggplot2

seaborn

simple_white

presentation

15. opacity

Transparency.

opacity=0.5

Range

0 → Fully Transparent

1 → Fully Visible

16. facet_row

Creates multiple graphs row-wise.

```
facet_row="Gender"
```

17. facet_col

Creates multiple graphs column-wise.

```
facet_col="City"
```

18. animation_frame

Creates animation.

```
animation_frame="Year"
```

19. animation_group

Groups animation objects.

```
animation_group="Country"
```

Most Common Plotly Express Functions

Function	Chart
px.scatter()	Scatter Plot
px.line()	Line Chart
px.bar()	Bar Chart
px.histogram()	Histogram
px.box()	Box Plot

Function	Chart
<code>px.violin()</code>	Violin Plot
<code>px.pie()</code>	Pie Chart
<code>px.area()</code>	Area Chart
<code>px.imshow()</code>	Heatmap/Image
<code>px.scatter_3d()</code>	3D Scatter
<code>px.line_3d()</code>	3D Line
<code>px.treemap()</code>	Treemap
<code>px.sunburst()</code>	Sunburst

update_layout()

Changes the overall chart layout.

```
fig.update_layout(
    title="Student Performance",
    width=900,
    height=500,
    xaxis_title="Age",
    yaxis_title="Marks"
)
```

Common update_layout() Attributes

Attribute	Purpose
<code>title</code>	Chart title
<code>width</code>	Width
<code>height</code>	Height
<code>template</code>	Theme
<code>showlegend</code>	Show/hide legend

Attribute	Purpose
legend_title	Legend title
paper_bgcolor	Outer background
plot_bgcolor	Plot area background
font_size	Font size
font_family	Font family
margin	Chart margins
xaxis_title	X-axis title
yaxis_title	Y-axis title

Example

```
fig.update_layout(
    title="Student Report",
    width=900,
    height=500,
    showlegend=True,
    plot_bgcolor="lightgray"
)
```

update_traces()

Changes the appearance of data points (markers, lines, bars, etc.).

```
fig.update_traces(
    marker_color="red",
    marker_size=15
)
```

Common update_traces() Attributes

Attribute	Purpose
marker_color	Marker color
marker_size	Marker size

Attribute	Purpose
marker_symbol	Marker shape
opacity	Transparency
line_color	Line color
line_width	Line width
textposition	Position of text labels
textfont_size	Text size

Example

```
fig.update_traces(
    marker_color="green",
    marker_size=18,
    opacity=0.7
)
```

Complete Example

```
import plotly.express as px
import pandas as pd
```

```
data = {
    "Name":["A","B","C","D"],
    "Age":[20,21,22,23],
    "Marks":[80,70,90,75],
    "City":["Delhi","Mumbai","Delhi","Noida"]
}
```

```
df = pd.DataFrame(data)
```

```
fig = px.scatter(
    df,
    x="Age",
    y="Marks",
    color="City",
    size="Marks",
    text="Name",
    hover_data=["City"],
    title="Student Performance",
    width=800,
```

```
    height=500,  
    template="plotly_dark"  
)  
  
fig.update_traces(  
    marker_size=20  
)  
  
fig.update_layout(  
    xaxis_title="Student Age",  
    yaxis_title="Marks Obtained"  
)  
  
fig.show()
```

Attributes You Will Use Most in Data Analysis

Attribute	Frequency
x	★★★★★
y	★★★★★
color	★★★★★
title	★★★★★
hover_data	★★★★★
labels	★★★★
template	★★★★
width	★★★
height	★★★
size	★★★
text	★★★
symbol	★★

Attribute	Frequency
-----------	-----------

facet_col	★ ★
-----------	-----

facet_row	★ ★
-----------	-----

animation_frame	★
-----------------	---

Sample Dataset

```
import pandas as pd
```

```
data = {  
    "Name":["A","B","C","D","E"],  
    "Age":[20,21,22,23,24],  
    "Marks":[80,65,90,70,95],  
    "City":["Delhi","Noida","Delhi","Mumbai","Mumbai"]  
}
```

```
df = pd.DataFrame(data)  
print(df)
```

Output

Name	Age	Marks	City
------	-----	-------	------

A	20	80	Delhi
---	----	----	-------

B	21	65	Noida
---	----	----	-------

C	22	90	Delhi
---	----	----	-------

D	23	70	Mumbai
---	----	----	--------

E	24	95	Mumbai
---	----	----	--------

1. Scatter Plot

```
import plotly.express as px
```

```
fig = px.scatter(df,  
    x="Age",  
    y="Marks",
```

```
title="Age vs Marks")
```

```
fig.show()
```

Explanation

- $x \rightarrow$ X-axis
 - $y \rightarrow$ Y-axis
 - title $\rightarrow$ Chart title
-

2. Line Chart

```
fig = px.line(df,  
              x="Age",  
              y="Marks",  
              title="Line Chart")
```

```
fig.show()
```

3. Bar Chart

```
fig = px.bar(df,  
             x="Name",  
             y="Marks",  
             title="Student Marks")
```

```
fig.show()
```

4. Horizontal Bar Chart

```
fig = px.bar(df,  
             x="Marks",  
             y="Name",  
             orientation='h')
```

```
fig.show()
```

5. Pie Chart

```
fig = px.pie(df,  
            names="City",  
            values="Marks",  
            title="Marks by City")
```

```
fig.show()
```

6. Histogram

```
fig = px.histogram(df,  
                   x="Marks",  
                   title="Marks Distribution")
```

```
fig.show()
```

7. Box Plot

```
fig = px.box(df,  
             y="Marks",  
             title="Box Plot")
```

```
fig.show()
```

Use

Detect Outliers

8. Violin Plot

```
fig = px.violin(df,  
               y="Marks")
```

```
fig.show()
```

9. Area Chart

```
fig = px.area(df,  
              x="Age",  
              y="Marks")
```

```
fig.show()
```

10. Bubble Chart

```
fig = px.scatter(df,  
                x="Age",  
                y="Marks",  
                size="Marks",
```

```
        color="City")

fig.show()
```

11. Color by Category

```
fig = px.scatter(df,
                 x="Age",
                 y="Marks",
                 color="City")

fig.show()
```

12. Facet Plot

```
fig = px.scatter(df,
                 x="Age",
                 y="Marks",
                 facet_col="City")

fig.show()
```

13. Sunburst Chart

```
fig = px.sunburst(df,
                  path=["City", "Name"],
                  values="Marks")

fig.show()
```

14. Treemap

```
fig = px.treemap(df,
                 path=["City", "Name"],
                 values="Marks")

fig.show()
```

15. Density Heatmap

```
fig = px.density_heatmap(df,
                         x="Age",
                         y="Marks")
```

```
fig.show()
```

16. Heatmap using Correlation

```
import plotly.express as px
```

```
corr = df.corr(numeric_only=True)
```

```
fig = px.imshow(corr,  
                text_auto=True,  
                color_continuous_scale="Viridis")
```

```
fig.show()
```

17. 3D Scatter Plot

```
fig = px.scatter_3d(df,  
                   x="Age",  
                   y="Marks",  
                   z="Age",  
                   color="City")
```

```
fig.show()
```

18. 3D Line Plot

```
fig = px.line_3d(df,  
                 x="Age",  
                 y="Marks",  
                 z="Age")
```

```
fig.show()
```

Customizing Charts

Change Color

```
fig = px.bar(df,  
             x="Name",  
             y="Marks",  
             color="City")
```

Change Template


```
fig = px.bar(df,  
             x="Name",  
             y="Marks",  
             template="plotly_dark")
```

Templates

```
plotly  
plotly_dark  
ggplot2  
seaborn  
presentation  
simple_white
```

Change Labels

```
fig = px.scatter(df,  
                 x="Age",  
                 y="Marks",  
                 labels={  
                     "Age": "Student Age",  
                     "Marks": "Exam Marks"  
                 })
```

Change Size

```
fig.update_layout(  
    width=900,  
    height=500  
)
```

Change Title

```
fig.update_layout(  
    title="Student Performance"  
)
```

Axis Labels

```
fig.update_layout(  
    xaxis_title="Age",  
    yaxis_title="Marks"  
)
```

Marker Size

```
fig.update_traces(  
    marker_size=15  
)
```

Marker Color

```
fig.update_traces(  
    marker_color="red"  
)
```

Line Width

```
fig.update_traces(  
    line_width=5  
)
```

Saving Plot

HTML

```
fig.write_html("chart.html")
```

PNG

```
fig.write_image("chart.png")
```

Install

```
pip install kaleido
```

Plotly in Streamlit

```
import streamlit as st  
import plotly.express as px  
import pandas as pd
```

```
data = {  
    "Name":["A","B","C","D"],  
    "Marks":[80,65,90,75]  
}
```

```
df = pd.DataFrame(data)
```

```
fig = px.bar(df,  
             x="Name",  
             y="Marks")
```

```
st.plotly_chart(fig)
```

Plotly vs Matplotlib

Feature	Plotly	Matplotlib
Interactive	✓	✗
Zoom	✓	✗
Hover Information	✓	✗
3D Charts	✓	✓
Dashboards	✓ Excellent	Limited
Streamlit Support	✓ Excellent	Good
Static Images	Good	Excellent
Learning Curve	Easy	Moderate